

Assignment 1: Forward Kinematics

16-384: Robot Kinematics and Dynamics

Prof. Jeff Ichnowski

Albert Xu

Henry Liao

Karthik Srinivasan

Contents

1	Code Questions	3
2	Hands-On Questions	7
3	Submission Checklist	9

1 Code Questions

In these problems you will implement the forward kinematics and workspace analysis you derived in the written section, and then watch your own code drive a simulated robot arm.

Setup

Copy the code handout to a location of your choice. Everything below is run from the top level of that folder — the one containing `local_autograder.py`.

The simulator, the visualizer and the collision checker all live in `xarm7_lib`, and the environment file there installs them:

```
conda env create -f xarm7_lib/environment.yml
conda activate 16384
```

The three exercises live in their own directories:

- `ex1` — forward kinematics of a planar RRR arm.
- `ex2` — workspace analysis of a PR arm and of an RRR arm.
- `ex3` — the demo videos you record in the hands-on question.

Testing your work

`local_autograder.py` checks your functions against a large set of stored test cases. Run all of them, or one exercise at a time:

```
python local_autograder.py          # everything
python local_autograder.py -p ex1   # just ex1
python local_autograder.py -p ex2   # just ex2
```

A failing run lists the first few cases that did not match; pass `--verbose` to see more. The stored cases live in `exN/inK.txt` (inputs) and `exN/outK.txt` (expected outputs).

A word of warning. The autograder on Gradescope does not use those stored outputs. It takes the same inputs, perturbs them by a small random amount that is different on every submission, and compares your result against a reference implementation. Code that reproduces the published numbers without actually computing the kinematics will score zero there. Write the general solution and the two graders will agree.

1) Forward Kinematics of an RRR Arm

30 points

Complete `ex1/forward_kinematics_RRR.py`. When implemented correctly, `forward_kinematics_RRR` computes the forward kinematics of a planar RRR arm from the three joint angles `theta1`, `theta2`, `theta3` and the three link lengths `l1`, `l2`, `l3`.

Each link lies along the x -axis of its starting frame, and each joint angle is measured counter-clockwise from the x -axis of the previous link. Because the arm is planar, every transform here is a 3×3 homogeneous matrix.

Seven frames are defined along the arm, alternating between a joint rotation and a translation down a link:

- $H_{1_0} = H_1^0$: the first joint, rotating $\{0\}$ by θ_1 to give $\{1\}$.
- $H_{2_1} = H_2^1$: the first link, translating $\{1\}$ by l_1 to give $\{2\}$.
- $H_{3_2} = H_3^2$: the second joint, rotating $\{2\}$ by θ_2 to give $\{3\}$.
- $H_{4_3} = H_4^3$: the second link, translating $\{3\}$ by l_2 to give $\{4\}$.
- $H_{5_4} = H_5^4$: the third joint, rotating $\{4\}$ by θ_3 to give $\{5\}$.
- $H_{6_5} = H_6^5$: the third link, translating $\{5\}$ by l_3 to give $\{6\}$.

$\{0\}$ is the base frame of the whole robot and $\{6\}$ is the end effector. Those six matrices are To-Do 1.

For To-Do 2, compose them into $H_{6_0} = H_6^0$, the transform taking a point expressed in the end effector frame $\{6\}$ into the base frame $\{0\}$:

$$H_6^0 = H_1^0 H_2^1 H_3^2 H_4^3 H_5^4 H_6^5$$

The two To-Dos are marked separately, so a correct H_{6_0} built from incorrect elementary transforms will not earn full marks — and neither will correct elementary transforms that are never composed.

To check your implementation:

- Start with angles of 0 and $\frac{\pi}{2}$, where you can work the answer out by hand, and confirm that each transform carries points from one link's frame into the previous one as you expect.
- Run `python local_autograder.py -p ex1`.
- See it move: `python local_autograder.py -p vis1` opens a browser with three sliders, one per joint angle. The arm follows the sliders, and the skeleton your `forward_kinematics_RRR` predicts is drawn on top of it. When your kinematics are right the two lie exactly on top of one another; when they are wrong, the gap between them is your error. The hands-on question below says more about this.

2) Workspace Analysis of a PR Arm

20 points

Move to the `ex2` directory. Here you will compute the reachable workspace of a PR (prismatic-rotary) arm by sweeping its joints over their full ranges.

Start by reading `workspace_analysis_RR.py`. It is a complete, working example for an RR arm: `endEffector_RR` places the end effector for one pair of joint angles, and `workspace_analysis_RR` sweeps both joints over `nSamples` values each and collects the `nSamples`² resulting positions. Compare the shape it produces to your analytical result from problem 8 of the written section. Use it as the template for everything that follows.

Your task is to complete `endEffector_PR` in `workspace_analysis_PR.py`. The sweep itself is already written for you, so this question is only the end effector position.

- `joints` is $[d, \theta]$: the prismatic extension d in metres and the revolute angle θ in radians.
- `linkLengths` is $[l_0, l_1]$. Note the convention here: l_0 is the base link length of the prismatic joint, so the prismatic joint reaches from l_0 out to $l_0 + d$, and l_1 is the length of the rotary link beyond it.
- The prismatic joint ranges from $d = 0$ to $d = d_f$, its fully extended length. That range is passed in as `minJoint[0]` and `maxJoint[0]`.

Check your implementation with `python local_autograder.py -p ex2`.

3) Workspace Analysis of an RRR Arm

30 points

Now complete `workspace_analysis_RRR.py`. Unlike the PR case, both functions in this file are yours to write.

- `endEffector_RRR(joints, linkLengths)` returns the (x, y) position of the end effector, where `joints` is $[\theta_1, \theta_2, \theta_3]$ and `linkLengths` is $[l_1, l_2, l_3]$. This is the translation part of the $\mathbf{H}_6^0$ you built in the forward kinematics question, and you should get the same answer both ways.
- `workspace_analysis_RRR(nSamples, minJoint, maxJoint, linkLengths)` sweeps all three joints. Each joint takes `nSamples` evenly spaced values between its entry in `minJoint` and its entry in `maxJoint`, and every combination is visited, so the returned `xs` and `ys` each hold `nSamples`³ entries.

The order in which you sweep the joints is up to you — the autograder compares the set of points you return, not the order they arrive in — but the number of points must be exactly `nSamples`³.

Check your implementation with `python local_autograder.py -p ex2`.

2 Hands-On Questions

1) Visualizing Your Kinematics on the xArm7

20 points

For this part of the homework you will run your own code on a simulated UFACTORY xArm7 and record what you see. This question is graded by hand from the videos you submit.

Why a 7-DoF arm can be your planar RRR arm

The xArm7 has seven revolute joints; the arm you have been deriving all assignment has three, in a plane. The two are the same mechanism once four of the seven joints are held still. Holding

$$\theta_2 = \frac{\pi}{2}, \quad \theta_3 = \frac{\pi}{2}, \quad \theta_5 = -\frac{\pi}{2}, \quad \theta_6 = \frac{\pi}{2}$$

leaves joints 1, 4 and 7 with their rotation axes all parallel to the world z -axis. Everything that can still move is then confined to a horizontal plane, and what is left is exactly a planar RRR arm: joint 1 at the shoulder, joint 4 at the elbow, joint 7 at the wrist. The visualizer measures l_1 , l_2 and l_3 off the robot model itself and hands those lengths to your functions, so the numbers you are tested on are the real geometry of the arm in front of you.

Recording the videos

Each command below opens a browser tab with the arm in it. Leave that tab visible and record your screen while it run. Any common format is fine (.mp4, .mov, .webm).

1. Forward kinematics.

```
python local_autograder.py -p vis1
```

Three sliders appear above the arm, one per joint angle, and the skeleton predicted by your `forward_kinematics_RRR` is drawn over the arm as you drag them. Record yourself moving all three sliders through a good part of their range. Your video should show the predicted skeleton staying on the arm throughout. If it drifts off, your kinematics are wrong — fix them before recording.

2. Workspace sweep.

```
python local_autograder.py -p vis2
```

The points your `workspace_analysis_RRR` predicts are drawn as a faint cloud, and the arm then walks through the same joint grid, lighting up each point as it reaches it. Record enough of the sweep that the traced points are clearly filling in the predicted cloud.

The joint limits for this sweep are chosen so that the arm never collides with itself, which is why the elbow angle does not use its full range.

Deliverables

Put both videos in the `ex3` directory; `create_submission.py` will pick up everything in there.

Run `create_submission.py` from the top level of the handout. It collects your three completed Python files and everything in `ex3`, and writes a single `handin.zip`:

```
python create_submission.py
```

This script does not grade your submission; it only packages it. Use `local_autograder.py` to check your answers before you submit.

3 Submission Checklist

- ☐ Complete `ex1/forward_kinematics_RRR.py`, `ex2/workspace_analysis_PR.py` and `ex2/workspace_analysis_RRR.py`.
- ☐ Confirm `python local_autograder.py` passes.
- ☐ Record the two demo videos and put them in `ex3`.
- ☐ Run `python create_submission.py` to produce `handin.zip`.
- ☐ Upload `handin.zip` to the **HW1 Code** assignment on Gradescope.